Primer entorno de desarrollo: terminal, Git y primer commit (Bloque 2) / Cómo armar en vivo un entorno completo, de cero a API

✓ **Hecho:** Ver

📘 **UNIDAD U1.2** · Setup técnico: terminal, Git, entorno Python y cliente HTTP

Cómo armar en vivo un entorno completo, de cero a API

Ya viste los conceptos por separado: **VS Code** como editor de referencia, la **terminal**, **Git**, **entornos virtuales** y **variables de entorno**.

Ahora toca verlos trabajar juntos, en orden, en un recorrido completo: desde una carpeta vacía hasta una petición autenticada respondiendo correctamente.

En esta sesión en vivo se arma este entorno de principio a fin, frente a un grupo. Esta lección recorre ese mismo camino paso a paso, para que puedas seguirlo, entenderlo, y usarlo como referencia cuando lo repliques tú en el laboratorio guiado de esta clase y en el bloque siguiente.

Antes de seguir: **Python** y **Git** ya deberías tenerlos instalados (los revisaste en la lección anterior de este mismo bloque). **Postman** o **Thunder** Client todavía no los necesitas instalar: vas a compararlos y elegir uno recién en el Bloque 4 de esta unidad; por ahora, solo obsérvalos en esta sesión en vivo.

🟢 El punto de partida: una carpeta vacía en VS Code

Todo recorrido empieza igual: una carpeta vacía en el computador, abierta en Visual Studio Code, el editor de referencia del programa.



Se abre en **VS Code** y se trabaja desde ahí usando la terminal integrada, sin salir del editor y sin usar atajos gráficos adicionales, precisamente para que quede claro qué hace cada comando. Este mismo orden (**carpeta**, **VS Code**, **terminal integrada**, y desde ahí todo lo demás) es el que vas a repetir cada vez que empieces un proyecto nuevo en este programa, no solo hoy.

📦 Del template al repositorio local

El primer paso es traer la **plantilla de repositorio** base del programa: una estructura de carpetas y archivos ya preparada, con las convenciones que se usan en todo el programa (dónde va el código, dónde la documentación, dónde los archivos de configuración).



Se descarga y se deja funcionando como un repositorio Git local, listo para su primer commit. Ese primer commit es intencionalmente pequeño: registra la estructura base tal como llegó, antes de modificar nada, para que el historial arranque desde un punto limpio y reconocible.

🧩 Entorno virtual y dependencias, en el mismo momento

Con el **repositorio ya iniciado**, el siguiente paso es crear el **entorno virtual** de **Python** para ese proyecto específico y activarlo.

Recién con el entorno activo se instalan las **primeras dependencias**: una librería para hacer peticiones **HTTP** y el cliente de la API del proveedor de modelo que se vaya a usar más adelante en el programa.

Apenas se instalan, esas dependencias quedan **registradas** en un archivo dentro del repositorio, para que el entorno completo sea reproducible por cualquier otra persona con un solo comando.

> **Este orden importa**: crear el entorno antes de instalar cualquier cosa evita que esas dependencias terminen mezcladas con las de otros proyectos del mismo computador.

En la sesión en vivo, esto se ve así en la terminal (no hace falta que lo repitas ahora: el bloque siguiente es

donde lo replicas tú mismo, paso a paso):

```
python -m venv venv
# activar el entorno según tu sistema operativo
pip install requests
pip install <cliente-de-tu-proveedor-de-modelo>
pip freeze > requirements.txt
```



El archivo que nunca se commitea

Antes de escribir cualquier credencial, se crea el archivo donde van a vivir las variables de entorno y, en el mismo paso, se agrega al archivo que le dice a Git qué ignorar.

El orden es deliberado: primero se excluye, después se crea el contenido sensible. Recién ahí se escribe, dentro de ese archivo ya excluido, la llave de API del proveedor de modelo como variable de entorno.

> **Un vistazo rápido al estado de Git confirma que ese archivo no aparece como algo por agregar**: si apareciera, sería la señal de que algo salió mal en el paso de exclusión, y habría que corregirlo antes de seguir.

Así se ve, en términos concretos, lo que se escribe en cada archivo (el detalle completo de esto lo practicas en el Bloque 4):

```
# en .gitignore
.env
# en .env
GEMINI_API_KEY=tu_llave_real_aquí
```

✅ La primera petición autenticada: cerrar el círculo

Con el **entorno** armado y la llave protegida, el último paso del recorrido es la prueba real: usando un cliente **HTTP** (**Postman** o **Thunder Client**), se arma una petición a un endpoint simple de la **API** del proveedor de modelo, autenticada con la llave que quedó guardada como variable de entorno, y se envía.

🟢 Una respuesta exitosa (**código 200** y un cuerpo con el resultado esperado) confirma que las piezas de la lección anterior (**VS Code**, **terminal**, **Git**, **entorno virtual** y **variables de entorno**) están funcionando juntas, no solo por separado. A modo de referencia, así se ve un resultado exitoso en términos generales (el formato exacto cambia según tu proveedor, tal como vas a ver en detalle en el Bloque 4):

```
Código de estado: 200
{
  "resultado": "contenido generado por el modelo..."
}
```

⚠️ **Si la petición falla, el error casi siempre señala directamente cuál de las piezas quedó mal configurada**: una llave mal copiada, un entorno no activado, o una dependencia no instalada (vas a ver estos códigos de error uno por uno en el Bloque 4, y la guía de troubleshooting por sistema operativo está más adelante en esta misma unidad).



🧭 Qué haces tú en esta misma clase, y qué queda para más adelante

Este recorrido completo es la referencia, no todavía tu tarea de hoy completa.

En el laboratorio guiado de esta misma clase vas a **inicializar tu propio repositorio** a partir de la plantilla base y hacer tu primer **commit**, que es exactamente el primer tramo de este recorrido.



La parte de **variables de entorno** y la primera petición autenticada las vas a practicar tú mismo en el **siguiente bloque**, con acompañamiento y con la guía de **troubleshooting** a mano por si algo no calza a la primera.

> 💡 **Es normal que la primera vez algo falle**: la mitad del valor de este recorrido es aprender a leer el error, casi siempre visible directamente en la terminal integrada de VS Code, y ubicar en cuál de los pasos ocurrió.

Siguiente actividad

Inicializa tu repositorio · Actividad práctica

Con este recorrido completo ya visto, pasa a la parte práctica de esta clase: inicializa tu propio repositorio a partir de la plantilla base del programa y registra tu primer **commit**.

Última modificación: jueves, 24 de septiembre de 2026, 10:46